

- What is Functional Interface?
- What is Lambda Expression?
- How to use Functional Interface with Lambda expression
- Advantage of Functional Interface?
- Types of Functional Interface?
 - o Consumer
 - o Supplier
 - o Function
 - o Predicate
- How to handle use case when Functional Interface extends from other Interface(or Functional Interface)?

}

What is Functional Interface:

- If an interface contains only 1 abstract method, that is known as Functional Interface.
- ✓ Also known as SAM Interface (Single Abstract Method).
- @FunctionalInterface keyword can be used at top of the interface (But its optional).

```
@FunctionalInterface
public interface Bird {
    void canFly(String val);
}
```

OR

```
public interface Bird {
    void canFly(String val);
}
```

```
@FunctionalInterface
public interface Bird {
    void canFly(String val);
    void getHeight();
}
```

<---- @FunctionalInterface Annotation restrict us and throws compilation error, if we try to add more than 1 abstract method

```
@FunctionalInterface
public interface Bird {
    void canFly(String val);
    default void getHeight(){
        //default method implementation
    }
    static void canEat(){
        //my static method implementation
    }
    String toString(); //Object class method
}
```

Abstract Method

<---- In Functional Interface, only 1 abstract method is allowed, but we can have other methods like default, static method or Methods inherited from the Object Class

Static

Object
↓
Class
do some thing
equal
habits

```
public interface TestInterface {
    String toString();
}
```

```
public class TestClassImplements implements TestInterface{
}
```

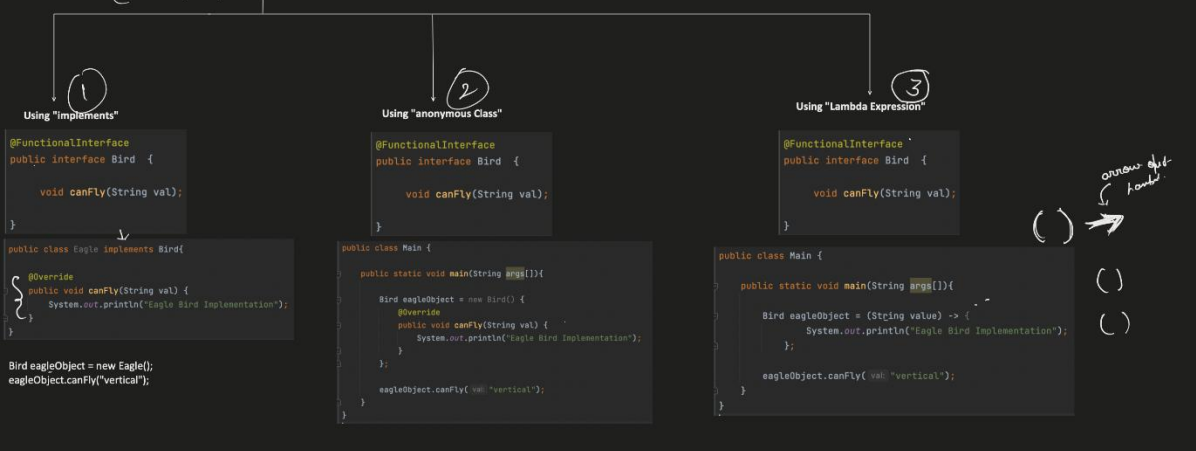
Object
do String
}

What is Lambda Expression:

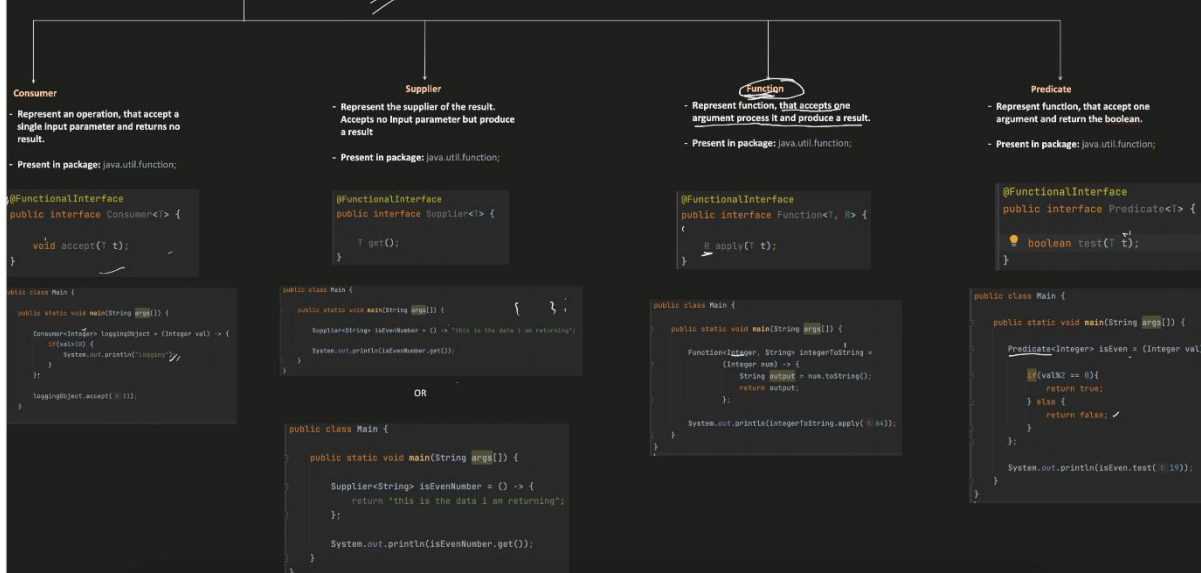
- Lambda expression is a way to implement the Functional Interface.

Before going into further into Lambda expression, let's first see:

Different Ways to Implements the Functional Interface:



Types of Functional Interface:



Handle use case when Functional Interface extends from other Interface:

Use Case 1: Functional Interface extending Non Functional Interface

```
public interface LivingThing {  
    public void canBreathe();  
}
```

```
@FunctionalInterface  
public interface Bird extends LivingThing {  
    void canFly(String val);  
}
```

```
public interface LivingThing {  
    default public boolean canBreathe(){  
        return true;  
    }  
}
```

```
@FunctionalInterface  
public interface Bird extends LivingThing {  
    void canFly(String val);  
}
```

Use Case 2: Interface extending Functional Interface

```
@FunctionalInterface  
public interface LivingThing {  
    public boolean canBreathe();  
}
```

```
public interface Bird extends LivingThing {  
    void canFly(String val);  
}
```

Use Case 3: Functional Interface extending other Functional Interface

```
✓ @FunctionalInterface
public interface LivingThing {

    public boolean canBreathe();
}
```

```
@FunctionalInterface
public interface Bird extends LivingThing {

    void canFly(String val); id
}
```

```
@FunctionalInterface
public interface LivingThing {

    public boolean canBreathe();
}
```

```
@FunctionalInterface
public interface Bird extends LivingThing {

    bool boolean canBreathe();
}
```

```
public class Main {

    public static void main(String args[]) {

        Bird eagle = () -> true;
        System.out.println(eagle.canBreathe());
    }
}
```

Bird obj = () -> true